Phase 5: UI/UX Documentation

Project Title: AI-Based In-Cabin Monitoring System

---

1. Dashboard Design & User Interface

The web dashboard is the primary interface for monitoring the vehicle cabin. It is built using Vanilla HTML5, CSS3 (Grid & Flexbox), and JavaScript, ensuring a lightweight footprint with zero heavy framework dependencies.

Design Principles

- **Dark Theme:** Chosen specifically for security monitoring environments to reduce driver eye strain during night shifts and to make color-coded threat alerts stand out visually.
- **Responsive Layout:** CSS Grid ensures the dashboard adapts fluidly across different screen sizes (from tablet monitors to desktop screens) without breaking the layout.
- **Minimalist Visuals:** Chart.js is used for plotting violence probability history dynamically without consuming excess CPU resources.

Core UI Components

1. **Live Video Feed:** Displays the real-time MJPEG stream from the cabin with bounding box annotations and an active FPS counter.
2. **Dynamic Threat Level Card:** Visual indicator reflecting the current system state (NORMAL, WATCH, WARNING, CRITICAL) using CSS box-shadow transitions (glow effects) for immediate attention.
3. **Violence Probability Meter:** A gradient bar transitioning from green to red, mapping the current VD-MIL output score.
4. **Alert History Feed:** A scrollable, slide-in animated feed logging timestamped events (e.g., "Phone Detected", "Drowsiness Alert", "Weapon Spotted").
5. **Statistics Grid:** Lifetime counters for total frames analyzed, total alerts per category, and active monitoring time.

Phase 6: Implementation Documentation

---

1. Source Code Structure

The repository is modularly structured to separate the backend API, AI inference engines, and frontend assets.

```
DEPI_Ai_Final_Project/
├── backend/
│   ├── app.py                # Main Flask server & API routing
│   ├── distraction.py        # YOLOv8n phone/cigarette detection module
│   ├── drowsiness.py         # YOLO11n + MediaPipe eye-state ensemble
│   ├── recognition.py        # ArcFace login & anti-spoofing
│   ├── violence.py           # VD-MIL + YOLOv8n threat evaluation
│   └── logger.py             # SQLite database interaction
├── frontend/
│   ├── index.html            # UI dashboard
│   ├── style.css             # CSS Grid and Dark Theme rules
│   └── script.js             # API polling and Chart.js logic
├── models/
│   ├── yolov8n_weapons.pt
│   ├── yolov8n_distraction.pt
│   ├── yolo11n_drowsiness.pt
│   └── movinet_violence/
├── data/
│   ├── users_db.json         # Authorized face embeddings
│   └── logs/                 # SQLite databases (events, activity)
└── requirements.txt          # Python dependencies
```

2. AI Model & Integration Workflow

Concurrency & Threading

The Flask web server (app.py) handles HTTP requests (REST API) and serves the UI. To prevent the heavy AI models from blocking the web server, the system utilizes **Background**

Threading.

- **Inference Loop:** A dedicated background thread captures webcam frames, passes them through the respective AI pipelines (e.g., YOLOv8 and MoViNet), and updates a globally shared state dictionary.
- **State Synchronization:** Python's Global Interpreter Lock (GIL) provides basic safety for single assignments, but threading.Lock() is implemented around critical sections (like updating the alert buffer or writing to SQLite) to prevent race conditions between the inference thread and the API polling thread.

AI Inference Pipelines

1. **Spatial Pipeline (Per-Frame):** YOLOv8n (Weapons/Distraction) and YOLO11n (Drowsiness) process individual frames to output bounding boxes and object classifications.
2. **Temporal Pipeline (Buffered):** VD-MIL maintains an 8-frame rolling buffer. The model processes the buffer using clean_activation_buffers() to ensure causal, real-time predictions without looking into the future.
3. **Geometric Pipeline:** MediaPipe extracts 468 facial landmarks to calculate EAR (Eye Aspect Ratio) and MAR (Mouth Aspect Ratio).

3. Coding Standards & Error Handling

- **Language & Conventions:** Python 3.11.5 following PEP8 standards. Functions are heavily modularized (e.g., separated bounding box drawing logic from inference logic).
- **Exception Handling:** Robust try-except blocks wrap the camera capture loops and model forward passes. If a frame drops or is corrupted, the system gracefully skips to the next frame without crashing the Flask server.
- **Stream Integrity:** Fixed MJPEG stream byte-concatenation to prevent SyntaxError breakages in the browser representation.
- **Security:** Face embeddings are stored locally. Anti-spoofing metrics (blur, brightness, texture, edge density) execute before the heavy ArcFace embedding extraction to prevent presentation attacks and save compute power.